Session 16 Teamwork — Make One Authorization Rule Reusable, Then Apply It Three More Times

Context

Work in your team (six teams of four students). Live code built one real, working, but deliberately **concrete** authorization guard: `AdminOrOwnerGuard` checks whether the signed-in user is a real Owner or Admin of the workspace named in the request body, and it gates exactly one route — `POST /projects`. It is hardcoded: one role set, one way of finding the workspace id, one route. This session still needs real authorization on four more real write routes (`PATCH` / `DELETE /projects/:id`, `PATCH` / `DELETE /workspaces/:id`), each with its **own** required role set, and two of them with the workspace id coming from a **different place in the request than the body**. Copy-pasting four near-identical guard classes is not the goal. Your team's job is to design and build a **reusable** mechanism — nobody has shown you how, live, today.

Goal

One shared authorization guard, configured per-route through **custom route metadata** you design and attach with a decorator, correctly protects all five real routes listed above — `POST /projects` (unchanged behavior) plus four more, each proven with real users of real, different roles.

What to build

Research, then build:

- **Custom route metadata.** Nest lets you attach arbitrary metadata to a route handler or a whole controller class, which a guard can read back at request time. Look up `SetMetadata` and how it is normally wrapped in your own custom decorator (a function that returns what `SetMetadata` produces). Design what your own metadata should carry: at minimum, which roles a route requires.
- **`Reflector`.** Look up `Reflector` and its `getAllAndOverride()` method — this is how a guard actually reads metadata your decorator attached, checking the specific route handler first and falling back to the whole controller class.
- **One shared guard**, replacing `AdminOrOwnerGuard`, that reads your metadata and performs the identical membership-and-role check `AdminOrOwnerGuard` already does — just driven by metadata instead of a hardcoded role array.
- **A real design decision your team must make and justify:** two of the four new routes (`PATCH` / `DELETE /projects/:id`) only have the **project's own `:id`** in their URL, not a workspace id anywhere in the request — your guard has to find the workspace a different way for these two than it does for `POST /projects`. Decide how your metadata/guard expresses "where do I find the workspace id for this particular route," and implement it. (`PATCH` / `DELETE /workspaces/:id`, by contrast, already have the workspace's own id directly in the URL — a simpler case, worth building first.)

- **Apply** your finished mechanism to all five routes with their real, correct role requirements, per `AUTH-AND-PERMISSIONS.md`:
  - `POST /projects` — Owner or Admin (same as live code; now expressed through your new mechanism).
  - `PATCH /projects/:id`, `DELETE /projects/:id` — Owner or Admin.
  - `PATCH /workspaces/:id` — Owner or Admin.
  - `DELETE /workspaces/:id` — **Owner only** (Admin is deliberately not enough here — re-check `AUTH-AND-PERMISSIONS.md` for why).
- **Required, written note:** your new shared guard will end up needing something from the `projects/` feature in order to work on `workspaces/` routes too. Find out what, and write down, in a short comment or team note, *why* — this is a real, disclosed architecture consequence of sharing one mechanism across two resources, not a mistake to hide.

Rules

Allowed: everything already taught live and in every prior session, plus whatever you discover by reading NestJS's own documentation on custom decorators, `SetMetadata`, and `Reflector` — these are real, standard Nest mechanisms, not something you have to invent from scratch. No new npm package is needed for any of this.

Not yet: no invite/add-member endpoint, no role-change endpoint, no endpoint that reads or lists `WorkspaceMember` rows directly — keep seeding membership facts for your own proofs directly via `mongosh`, exactly like live code did. No changes to Issues (that is this session's own homework). No changes to `GET /projects/stats`'s own guard.

Test it (design your own full proof — this is a starting point, not the whole list)

BASH

```
# register and log in at least three real users; make one the real Owner of a real workspace
# (by creating it), then seed the other two directly into workspacemembers as 'admin' and
'member'

# for POST /projects and PATCH/DELETE /projects/:id and PATCH/DELETE /workspaces/:id:
# prove a plain member is rejected, prove the required role succeeds

curl -i -X PATCH http://localhost:3000/projects/<a real id> \
  -H "Authorization: Bearer <the plain member's token>" -H "Content-Type: application/json"
-d '{}'

curl -i -X DELETE http://localhost:3000/workspaces/<a real id> \
  -H "Authorization: Bearer <a real admin's token>"
# this one must FAIL -- Admin is not enough to delete a Workspace

curl -i -X DELETE http://localhost:3000/workspaces/<a real id> \
  -H "Authorization: Bearer <the real owner's token>"
# this one must SUCCEED
```

Also re-run live code's own earlier proof against `POST /projects` and confirm nothing changed — your new mechanism must reproduce its exact prior behavior, not merely add new routes.

Done when

- One shared guard, configured through your own custom route metadata, protects all five routes listed above — no second near-duplicate guard class exists anywhere in the codebase.
- `POST /projects`'s own real behavior is unchanged, proven by re-running live code's own proof.
- `PATCH` / `DELETE /projects/:id` correctly require Owner or Admin, with the workspace resolved through the project itself (not from the request body).
- `PATCH /workspaces/:id` correctly requires Owner or Admin; `DELETE /workspaces/:id` correctly requires Owner **only** — proven with a real Admin correctly rejected and a real Owner correctly succeeding on the same route.
- The required written note about the new real module dependency your shared guard introduced exists and correctly names *why*, not just *that*.
- `GET /projects/stats` and every previously working behavior in this app are unaffected, proven, not assumed.
- Every team member can explain, unprompted, what `SetMetadata` / `Reflector` actually do and why a second, hardcoded guard class was not the right next step.

Working time

About 65–75 minutes — this introduces a genuinely new Nest mechanism nobody has shown you live today; budget real time to read, experiment, and debug, not just to type.